

Hyperparameter Tuning an Artificial Neural Network with Optuna

Dynamic PyTorch Models, Adaptive Search, and Experiment Tracking

Abstract

Neural-network weights are learned by gradient-based training, but choices such as hidden depth, hidden width, epoch count, learning rate, batch size, optimizer, dropout probability, and weight decay configure that training process. These notes develop an eight-dimensional Optuna search for a Fashion-MNIST artificial neural network. The treatment begins with the distinction between parameters and hyperparameters, derives the validation objective and dynamic tensor shapes, works through a two-hidden-layer numerical example, constructs the model and loaders dynamically, and maps every sampled value to the object it controls. Two rounds of tuning are reconstructed: an architecture-only search and a complete search over all eight controls. The final sections explain validation discipline, trial independence, reproducibility, common implementation failures, and how MLflow-style experiment tracking supports careful interpretation without turning marginal plots into causal claims.

Contents

1	Purpose and complete mental map	2
1.1	The practical problem	2
2	The eight-dimensional search space	3
2.1	How batch size changes computation	4
2.2	Grid, random, and adaptive search	4
3	Data, tensor, and evaluation contracts	5
3.1	Prepare the Fashion-MNIST table	5
3.2	Define the index-to-example mapping	6
4	Constructing a dynamic artificial neural network	6
4.1	Why the architecture must be dynamic	6
4.2	Expanded construction example	6
4.3	Parameter count for the manual example	8
5	The objective function as a complete experiment	9
5.1	Reusable training and evaluation operations	9
6	Round one: tune architecture only	10
6.1	Sample depth and width	10
6.2	Create one independent round-one trial	11
6.3	Create and run a study	11

7	Round two: tune all eight controls	12
7.1	Define each sampling rule	12
7.2	Why learning rate and weight decay use a log scale	12
7.3	Move batch-dependent loaders inside the objective	12
7.4	Map an optimizer name to a fresh optimizer	13
7.5	Assemble the complete objective	14
8	Interpreting the two search rounds	14
8.1	What a best trial does and does not establish	14
8.2	Ways to improve the search	15
9	Validation discipline and unbiased testing	15
10	Experiment tracking with MLflow-style runs	16
11	Executed reconstruction and reproducibility	17
12	Failure modes and debugging checklist	18
13	Review questions	18

1 Purpose and complete mental map

1.1 The practical problem

Fashion-MNIST is a ten-class image-classification collection. Each grayscale image has height 28 pixels and width 28 pixels. Flattening one image produces

$$28 \times 28 = 784$$

input features. An artificial neural network (ANN) maps a mini-batch

$$X \in \mathbb{R}^{B \times 784}$$

to logits

$$Z \in \mathbb{R}^{B \times 10},$$

where B is the mini-batch size and each row of Z contains ten unnormalized class scores.

Earlier configurations used two hidden layers with widths 128 and 64, 100 epochs, batch size 32, a manually chosen learning rate, dropout probability 0.3, and a fixed weight-decay coefficient. Those values produced useful accuracy, but usefulness does not make them uniquely correct. A different architecture or fitting configuration may generalize better, train faster, consume less memory, or behave more reliably.

Definition

A **parameter** is a value learned inside one training run, such as a weight or bias. A **hyperparameter** is a value that configures the architecture, data pipeline, optimizer, regularization, or training schedule and is selected outside the weight-update loop.

Let h denote one complete hyperparameter configuration. Training with h produces fitted parameters $\theta^*(h)$. For a validation set with N_{val} examples, validation accuracy is

$$A_{\text{val}}(h) = \frac{1}{N_{\text{val}}} \sum_{i=1}^{N_{\text{val}}} \mathbf{1} \left[\arg \max_{c \in \{0, \dots, 9\}} f_{\theta^*(h)}(x_i)_c = y_i \right].$$

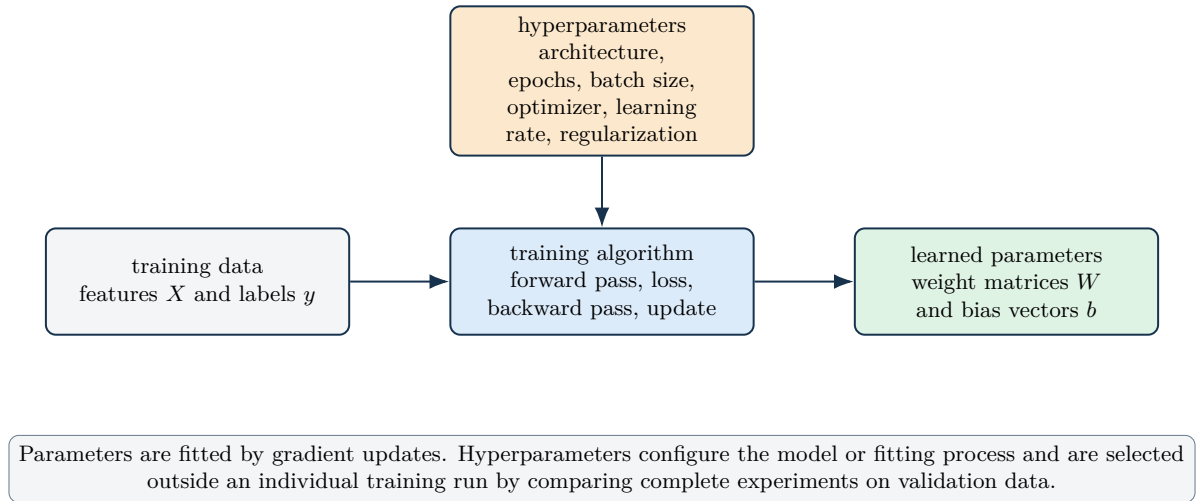


Figure 1: Parameters are fitted within a run, whereas hyperparameters configure and compare complete runs. [Technical reference](#).

Every symbol has a specific role: x_i is one validation input, y_i is its integer class label, $f_{\theta^*(h)}(x_i)_c$ is the logit for class c , and $\mathbf{1}[\cdot]$ equals 1 when the prediction is correct and 0 otherwise. The outer selection problem is

$$h^* \in \arg \max_{h \in \mathcal{H}} A_{\text{val}}(h),$$

where $\mathcal{H}$ is the search space. This equation asks for the tested configuration with the largest validation score. It does not guarantee that a finite search finds the global optimum.

Intuition

Gradient descent adjusts millions of weights within one run. Hyperparameter tuning compares many entire runs. The outer procedure is expensive because every score may require constructing, training, and evaluating a new model.

Takeaway

A configuration chosen only by intuition is a starting point. A defensible final choice requires an explicit search space, a repeatable objective, validation data, and honest reporting of the search budget.

2 The eight-dimensional search space

The complete search controls eight quantities. The ranges below are not universal constants; they are bounded hypotheses for this task.

Control	Search rule	Responsibility
Hidden layers L	integers 1 through 5	network depth and representational capacity
Neurons per layer H	8, 16, ..., 128	equal width of every hidden layer
Epochs E	10, 20, 30, 40, 50	complete passes over the training partition
Learning rate η	10^{-5} through 10^{-1} , log scale	optimizer step magnitude
Dropout probability p	0.1, 0.2, ..., 0.5	probability of masking a hidden activation during training
Batch size B	16, 32, 64, 128	examples processed before one optimizer step
Optimizer family	Adam, SGD, RMSprop	parameter-update rule
Weight decay λ	10^{-5} through 10^{-3} , log scale	shrinkage strength supplied to the optimizer

The architecture-only search has five possible depths and sixteen possible widths:

$$5 \times \frac{128 - 8}{8} + 5 = 5 \times 16 = 80$$

discrete architecture templates. The full search has at least

$$5 \times 16 \times 5 \times 5 \times 4 \times 3 = 24,000$$

discrete combinations before the continuously sampled learning-rate and weight-decay values are considered. Ten trials therefore inspect only a tiny part of $\mathcal{H}$.

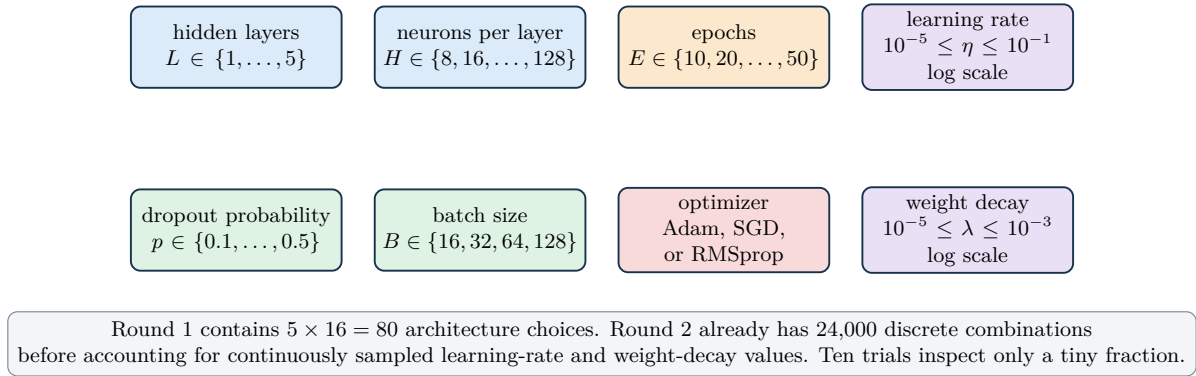


Figure 2: Eight controls form a mixed search space containing integers, ordered grids, categories, and log-scaled continuous values. [Technical reference](#).

2.1 How batch size changes computation

With 48,000 training examples and no incomplete batch, the number of optimizer updates per epoch is

$$U_{\text{epoch}} = \frac{48,000}{B}.$$

For the four candidate batch sizes, the counts are

$$B = 16 \Rightarrow 3000, \quad B = 32 \Rightarrow 1500, \quad B = 64 \Rightarrow 750, \quad B = 128 \Rightarrow 375.$$

Thus batch size affects memory, gradient noise, BatchNorm statistics, and update count. Comparing configurations at equal epoch counts does not compare equal numbers of parameter updates.

2.2 Grid, random, and adaptive search

A grid enumerates predetermined combinations. Random search draws independent combinations and often covers high-dimensional spaces more efficiently than a rigid grid. Optuna’s default sampler for a single-objective study is the tree-structured Parzen estimator (TPE) sampler. TPE is an adaptive method: completed outcomes influence later proposals.

In a simplified TPE interpretation, observed configurations are divided into relatively good and relatively poor groups. The sampler estimates a density $\ell(h)$ for good configurations and a density $g(h)$ for the others, then favors candidates with a large ratio

$$\frac{\ell(h)}{g(h)}.$$

The density notation is conceptual: ℓ and g summarize where useful and less-useful observations have appeared. Early trials still need exploration because no reliable history exists yet.

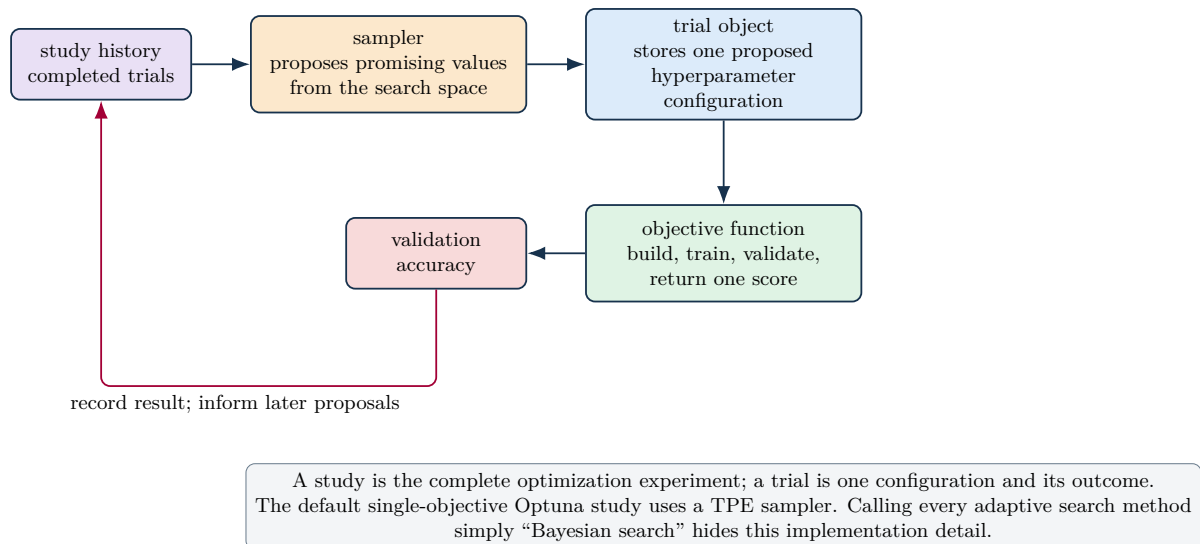


Figure 3: A study records completed trials so that an adaptive sampler can propose later configurations. [Technical reference](#).

Pitfall

Calling every adaptive search procedure “Bayesian search” hides important implementation choices. Sampler class, seed, startup behavior, pruning policy, and Optuna version all affect the sequence of proposed trials.

3 Data, tensor, and evaluation contracts

3.1 Prepare the Fashion-MNIST table

The available table contains 60,000 rows. Pixel values are scaled from integer intensities in $[0, 255]$ to floating-point values in $[0, 1]$. The repeatedly consulted held-out partition must be treated as validation data during tuning.

```

import pandas as pd
from sklearn.model_selection import train_test_split

frame = pd.read_csv("fashion-mnist_train.csv")
X = frame.iloc[:, 1:].to_numpy(dtype="float32")
y = frame.iloc[:, 0].to_numpy(dtype="int64")

X_train, X_valid, y_train, y_valid = train_test_split(
    X, y, test_size=0.2, random_state=42
)
X_train = X_train / 255.0
X_valid = X_valid / 255.0

assert X_train.shape == (48_000, 784)
assert X_valid.shape == (12_000, 784)
assert y_train.shape == (48_000,)
assert y_valid.shape == (12_000,)

```

Listing 1: Load, split, and scale the Fashion-MNIST table

The target shape is $[N]$, not $[N, 1]$. Cross-entropy expects one integer class index per row. Scaling changes numerical range but not tensor shape.

3.2 Define the index-to-example mapping

A PyTorch `Dataset` owns examples and maps an integer index to one aligned feature-label pair. Its invariant is

$$\text{len}(\text{features}) = \text{len}(\text{labels}).$$

```
import torch
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, features, labels):
        self.features = torch.tensor(
            features, dtype=torch.float32
        )
        self.labels = torch.tensor(
            labels, dtype=torch.long
        )

    def __len__(self):
        return len(self.features)

    def __getitem__(self, index):
        return self.features[index], self.labels[index]

train_dataset = CustomDataset(X_train, y_train)
valid_dataset = CustomDataset(X_valid, y_valid)
```

Listing 2: Wrap aligned feature and label arrays in a custom dataset

For batch size B , a loader produces features shaped $[B, 784]$ and labels shaped $[B]$. The model must return logits shaped $[B, 10]$.

Shape and state check

The data loader controls mini-batch grouping; it does not move tensors to a GPU. Every feature batch, label batch, and model parameter must be on the same device before the forward pass.

4 Constructing a dynamic artificial neural network

4.1 Why the architecture must be dynamic

A fixed `nn.Sequential` definition cannot represent trials with one through five hidden layers. The constructor must build a module list from L , H , and p , then unpack that list into `nn.Sequential`.

For one linear layer with input width d_{in} and output width d_{out} , a batch computation is

$$Y = XW^T + \mathbf{1}b^T.$$

Here $X \in \mathbb{R}^{B \times d_{\text{in}}}$, $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, $b \in \mathbb{R}^{d_{\text{out}}}$, and $Y \in \mathbb{R}^{B \times d_{\text{out}}}$. The vector $\mathbf{1} \in \mathbb{R}^B$ repeats the same bias for all examples.

For $L = 2$ and $H = 128$, the shape flow is

$$[B, 784] \rightarrow [B, 128] \rightarrow [B, 128] \rightarrow [B, 10].$$

BatchNorm, ReLU, and dropout preserve each hidden tensor's shape.

4.2 Expanded construction example

Initialize the current input width as $d = 784$.

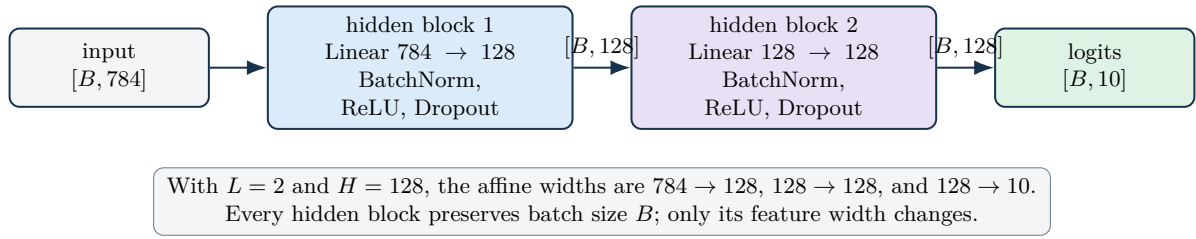


Figure 4: A two-hidden-layer sampled configuration preserves batch size while changing feature width. [Technical reference](#).

1. First iteration: append `Linear(784, 128)` and set $d \leftarrow 128$.
2. Second iteration: append `Linear(128, 128)` and keep $d = 128$.
3. After the loop: append `Linear(128, 10)`.

The mutation $d \leftarrow H$ is essential. Without it, the second hidden layer incorrectly expects 784 inputs even though the preceding layer emits 128 features.

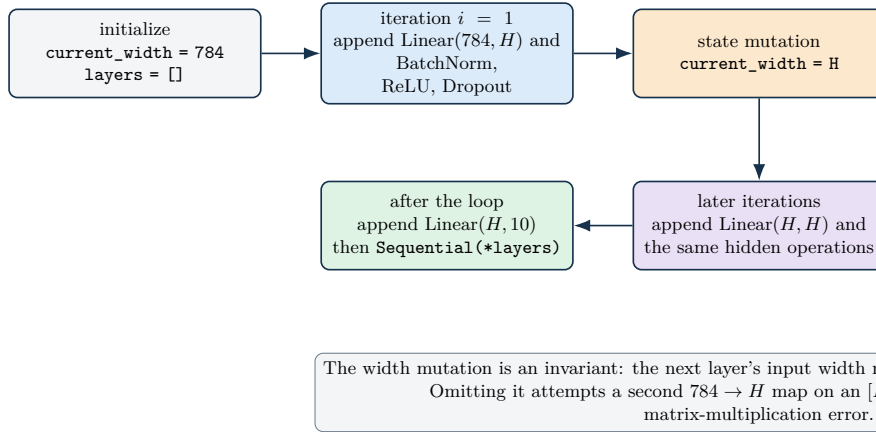


Figure 5: The builder updates its current width after every hidden layer before appending the output layer. [Technical reference](#).

```

import torch.nn as nn

class DynamicANN(nn.Module):
    def __init__(
        self, input_dim, output_dim,
        num_hidden_layers, neurons_per_layer,
        dropout_rate,
    ):
        super().__init__()
        layers = []
        current_width = input_dim

        for _ in range(num_hidden_layers):
            layers.append(
                nn.Linear(current_width, neurons_per_layer)
            )
            layers.append(nn.BatchNorm1d(neurons_per_layer))
            layers.append(nn.ReLU())
            layers.append(nn.Dropout(p=dropout_rate))
            current_width = neurons_per_layer

        layers.append(nn.Linear(current_width, output_dim))
        self.model = nn.Sequential(*layers)

    def forward(self, features):
        return self.model(features)

```

Listing 3: Build an equal-width ANN from sampled architecture controls

4.3 Parameter count for the manual example

With BatchNorm in every hidden block, the trainable parameter count is

$$\begin{aligned}
 P(L, H) = & (784H + H) \\
 & + (L - 1)(H^2 + H) \\
 & + (10H + 10) \\
 & + 2LH.
 \end{aligned}$$

The first line counts the input linear layer. The second counts the remaining hidden linear layers. The third counts the output layer. The fourth counts one trainable BatchNorm scale and shift for each hidden feature.

For $L = 2$ and $H = 128$,

$$\begin{aligned}
 P(2, 128) &= (784 \cdot 128 + 128) + (128^2 + 128) \\
 &\quad + (128 \cdot 10 + 10) + 2(2)(128) \\
 &= 100,480 + 16,512 + 1,290 + 512 \\
 &= 118,794.
 \end{aligned}$$

The executed PyTorch model produced exactly 118,794 trainable scalars and logits shaped $[32, 10]$ for an input shaped $[32, 784]$. BatchNorm running means and variances are buffers, not trainable parameters, and are therefore absent from P .

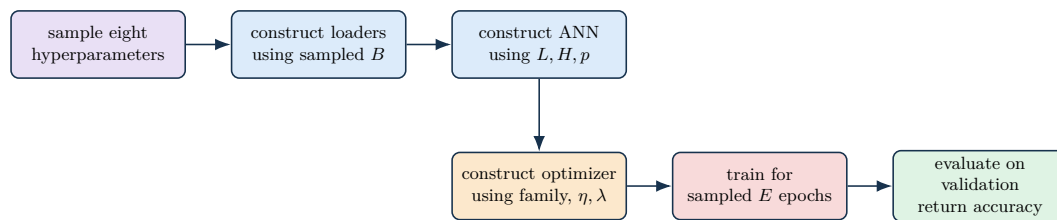
Pitfall

Every trial must create a new model. Reusing one model across trials leaks fitted weights and BatchNorm running statistics from earlier configurations, making trial scores incomparable.

5 The objective function as a complete experiment

An Optuna objective receives one `Trial` object and returns one scalar score. Its responsibilities are ordered:

1. sample a configuration from the search space;
2. create objects affected by that configuration;
3. create a fresh model, loss, and optimizer;
4. train for the selected schedule;
5. evaluate on validation data;
6. return validation accuracy.



Objects affected by sampled values must be built inside the objective. A loader created outside cannot change batch size, and a reused model would carry trained weights from one trial into the next. Every trial must begin independently.

Figure 6: Every sampled value must reach the data, model, optimizer, or training object that it controls. [Technical reference](#).

5.1 Reusable training and evaluation operations

The fitting function activates training state so dropout samples masks and BatchNorm updates running statistics. The evaluation function activates inference state and disables gradient recording.

```

def fit(model, loader, criterion, optimizer, epochs, device):
    for _epoch in range(epochs):
        model.train()
        for features, targets in loader:
            features = features.to(device)
            targets = targets.to(device)

            optimizer.zero_grad(set_to_none=True)
            logits = model(features)
            loss = criterion(logits, targets)
            loss.backward()
            optimizer.step()
  
```

Listing 4: Fit a model for a selected number of epochs

```
def accuracy(model, loader, device):
    model.eval()
    correct = 0
    total = 0

    with torch.inference_mode():
        for features, targets in loader:
            features = features.to(device)
            targets = targets.to(device)
            predictions = model(features).argmax(dim=1)
            correct += (predictions == targets).sum().item()
            total += targets.numel()

    return correct / total
```

Listing 5: Return deterministic validation accuracy

`model.eval()` and `torch.inference_mode()` have different responsibilities. The first changes Dropout and BatchNorm behavior. The second disables autograd bookkeeping. Correct validation ordinarily needs both.

6 Round one: tune architecture only

6.1 Sample depth and width

The first round varies hidden depth and equal hidden width while holding the remaining values fixed. `suggest_int` can represent both ranges.

```
def sample_architecture(trial):
    num_hidden_layers = trial.suggest_int(
        "num_hidden_layers", 1, 5
    )
    neurons_per_layer = trial.suggest_int(
        "neurons_per_layer", 8, 128, step=8
    )
    return num_hidden_layers, neurons_per_layer
```

Listing 6: Sample the two architecture hyperparameters

The names are study keys. Optuna stores each returned value under its name so that `best_params` can later reconstruct the winning configuration.

6.2 Create one independent round-one trial

```
def objective_round_one(trial):
    depth, width = sample_architecture(trial)

    model = DynamicANN(
        input_dim=784,
        output_dim=10,
        num_hidden_layers=depth,
        neurons_per_layer=width,
        dropout_rate=0.3,
    ).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = torch.optim.SGD(
        model.parameters(), lr=0.1, weight_decay=1e-4
    )
    fit(
        model, train_loader, criterion,
        optimizer, epochs=50, device=device
    )
    return accuracy(model, valid_loader, device)
```

Listing 7: Run one architecture-only objective

The function does not print or return the model. It returns exactly one floating-point validation accuracy because the study direction is to maximize that score.

The displayed cell also retained a variable assignment `learning_rate = 0.01` while the optimizer call directly supplied `lr=0.1`. The variable therefore had no effect. The reconstruction uses the optimizer's effective value, 0.1, and removes the stale assignment. A configuration variable should be passed into the object it is intended to control; otherwise editing it creates a silent false experiment.

6.3 Create and run a study

```
import optuna

study = optuna.create_study(direction="maximize")
study.optimize(objective_round_one, n_trials=10)

best_accuracy = study.best_value
best_architecture = study.best_params
```

Listing 8: Create a maximizing study and execute ten trials

`study` is the complete experiment. `study.optimize` repeatedly creates a trial, calls the objective, and records the returned score. When the score is accuracy, `direction="maximize"` is correct. A loss objective ordinarily uses `direction="minimize"`.

The architecture-only run displayed a best value of 0.8933333333333333, or 89.3333%, with three hidden layers and 120 neurons per hidden layer. This result means that configuration was best among the ten executed trials. It does not mean all 80 architecture templates were evaluated.

Pitfall

The keyword is `n_trials`, not `n_trial`. A cell defining the objective must also run before `study.optimize` can call it. The displayed workflow encountered both an undefined-objective error and the singular-keyword error before correction.

7 Round two: tune all eight controls

7.1 Define each sampling rule

```
def sample_configuration(trial):
    return {
        "num_hidden_layers": trial.suggest_int(
            "num_hidden_layers", 1, 5
        ),
        "neurons_per_layer": trial.suggest_int(
            "neurons_per_layer", 8, 128, step=8
        ),
        "epochs": trial.suggest_int(
            "epochs", 10, 50, step=10
        ),
        "learning_rate": trial.suggest_float(
            "learning_rate", 1e-5, 1e-1, log=True
        ),
        "dropout_rate": trial.suggest_float(
            "dropout_rate", 0.1, 0.5, step=0.1
        ),
        "batch_size": trial.suggest_categorical(
            "batch_size", [16, 32, 64, 128]
        ),
        "optimizer_name": trial.suggest_categorical(
            "optimizer", ["Adam", "SGD", "RMSprop"]
        ),
        "weight_decay": trial.suggest_float(
            "weight_decay", 1e-5, 1e-3, log=True
        ),
    }
```

Listing 9: Sample the complete hyperparameter configuration

7.2 Why learning rate and weight decay use a log scale

A log-uniform learning-rate sample can be described by

$$u \sim \text{Uniform}(\log 10^{-5}, \log 10^{-1}), \quad \eta = e^u.$$

Equal intervals in u represent equal multiplicative factors in η . The geometric midpoint is

$$\sqrt{10^{-5} \cdot 10^{-1}} = \sqrt{10^{-6}} = 10^{-3}.$$

This midpoint is multiplicatively centered: it is 100 times above 10^{-5} and 100 times below 10^{-1} .

Pitfall

Optuna does not permit a linear `step` together with `log=True`. Use logarithmic continuous sampling, or explicitly provide a categorical list of desired powers and multipliers.

7.3 Move batch-dependent loaders inside the objective

A loader created once outside the objective cannot change when the trial samples a new batch size. Both loaders must therefore be constructed after sampling B .

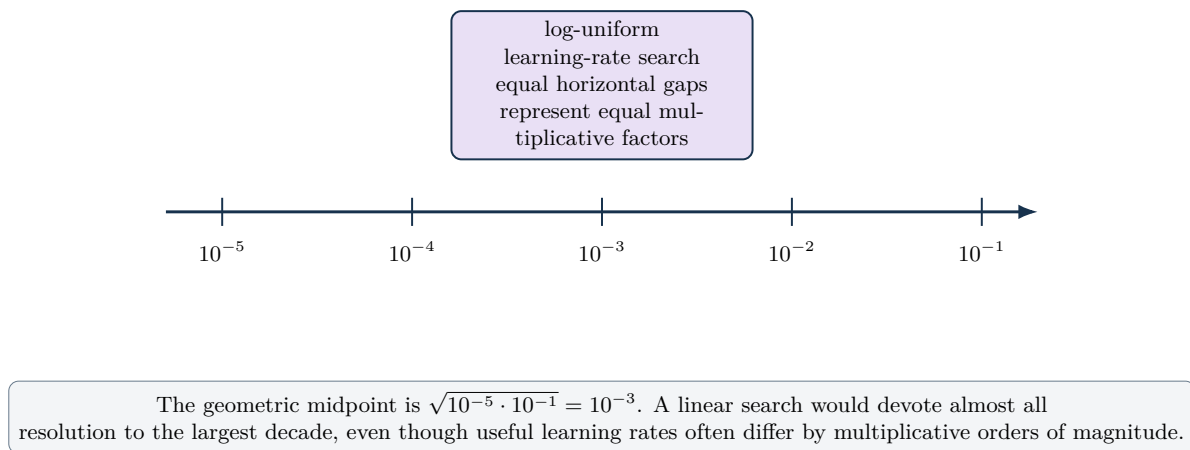


Figure 7: A logarithmic scale allocates comparable search resolution to every order of magnitude. [Technical reference.](#)

```
from torch.utils.data import DataLoader

def make_loaders(batch_size):
    train_loader = DataLoader(
        train_dataset,
        batch_size=batch_size,
        shuffle=True,
        pin_memory=(device.type == "cuda"),
    )
    valid_loader = DataLoader(
        valid_dataset,
        batch_size=batch_size,
        shuffle=False,
        pin_memory=(device.type == "cuda"),
    )
    return train_loader, valid_loader
```

Listing 10: Create loaders from the sampled batch size

Pinned host memory can accelerate transfer to CUDA when batches are moved with a compatible non-blocking copy. It does not move tensors by itself and offers no GPU-transfer benefit on a CPU-only run, so the safer expression enables it only for a CUDA device.

7.4 Map an optimizer name to a fresh optimizer

```
def make_optimizer(name, model, learning_rate, weight_decay):
    optimizer_classes = {
        "Adam": torch.optim.Adam,
        "SGD": torch.optim.SGD,
        "RMSprop": torch.optim.RMSprop,
    }
    optimizer_class = optimizer_classes[name]
    return optimizer_class(
        model.parameters(),
        lr=learning_rate,
        weight_decay=weight_decay,
    )
```

Listing 11: Construct the sampled optimizer family

The optimizer owns state associated with a particular model's parameters. It must be recreated with every fresh model. Adam and RMSprop maintain moving statistics; carrying them across trials would be another form of leakage.

7.5 Assemble the complete objective

```
def objective(trial):
    config = sample_configuration(trial)
    train_loader, valid_loader = make_loaders(
        config["batch_size"]
    )

    model = DynamicANN(
        input_dim=784,
        output_dim=10,
        num_hidden_layers=config["num_hidden_layers"],
        neurons_per_layer=config["neurons_per_layer"],
        dropout_rate=config["dropout_rate"],
    ).to(device)

    criterion = nn.CrossEntropyLoss()
    optimizer = make_optimizer(
        config["optimizer_name"],
        model,
        config["learning_rate"],
        config["weight_decay"],
    )
    fit(
        model, train_loader, criterion,
        optimizer, config["epochs"], device
    )
    return accuracy(model, valid_loader, device)
```

Listing 12: Execute one complete eight-parameter trial

The same study controller can now execute ten full trials:

```
study = optuna.create_study(direction="maximize")
study.optimize(objective, n_trials=10)

print(study.best_value)
print(study.best_params)
```

Listing 13: Optimize the complete objective and inspect the best trial

The method name is `suggest_categorical`. The displayed misspelling `suggest_categorical` raises an attribute error before training begins.

8 Interpreting the two search rounds

The second ten-trial search displayed a best value of 0.8908333333333334, or 89.0833%. Its best configuration used four hidden layers, 88 neurons per hidden layer, 20 epochs, learning rate 0.090300318300747871, dropout probability 0.2, batch size 128, RMSprop, and weight decay 0.0004539602814543209. The spoken summary near 89.7% does not match the exact displayed value and is therefore not used for arithmetic. The selected optimizer also differed from the stronger SGD pattern later observed in a larger tracked search; finite adaptive searches can return different winners.

8.1 What a best trial does and does not establish

The study attribute `best_value` equals the maximum recorded score when the direction is maximize. The attribute `best_params` stores the associated sampled configuration. These statements support four conclusions:

1. the trial executed successfully;
2. its validation score exceeded the other completed scores in that study;

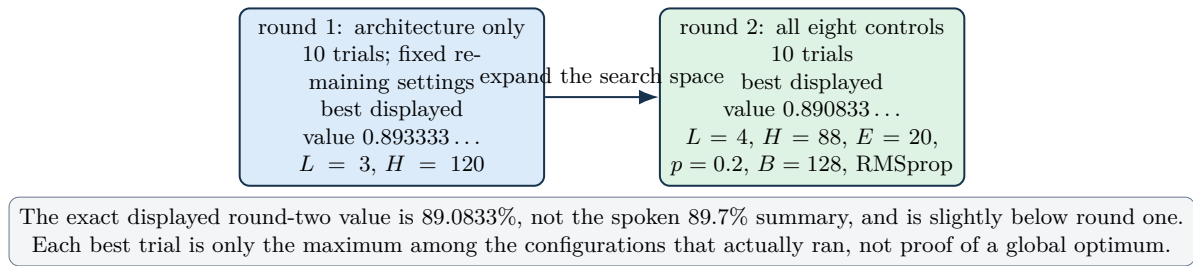


Figure 8: The exact displayed complete-search score was slightly below the architecture-only score; neither ten-trial study exhausts its search space. [Technical reference](#).

3. the stored values reconstruct that trial’s hyperparameters;
4. a larger budget or changed search space may produce a different best trial.

They do not prove that the configuration is globally optimal, that every hyperparameter independently caused improvement, or that the validation score is an unbiased final test estimate.

8.2 Ways to improve the search

- Increase the trial budget from 10 to 50 or more when compute permits.
- Widen a bound only when useful values appear near that bound or domain knowledge supports exploration.
- Add other optimizer families only with appropriate learning-rate and decay semantics.
- Use pruning to stop clearly unpromising trials before their full epoch count.
- Repeat important configurations across random seeds.
- Record runtime and memory as additional objectives or constraints when deployment cost matters.

An optional pruning pattern reports an intermediate validation score:

```
for epoch in range(config["epochs"]):
    train_one_epoch(model, train_loader, criterion, optimizer, device)
    value = accuracy(model, valid_loader, device)
    trial.report(value, step=epoch)
    if trial.should_prune():
        raise optuna.TrialPruned()
```

Listing 14: Report intermediate scores and prune unpromising trials

Pruning saves compute, but it also changes which configurations receive complete training. The pruner and intermediate metric must therefore be recorded as part of the experimental design.

9 Validation discipline and unbiased testing

If a held-out loader repeatedly determines architecture, optimizer, learning rate, or any other choice, that loader is validation data regardless of whether a variable calls it `test_loader`. Reusing it for selection makes its best score optimistic.

A stronger split uses three disjoint partitions:

Training partition updates weights, biases, optimizer state, and BatchNorm running statistics.

Validation partition returns the objective score and selects the best trial.

Test partition is evaluated once after the study, architecture, and retraining procedure are fixed.

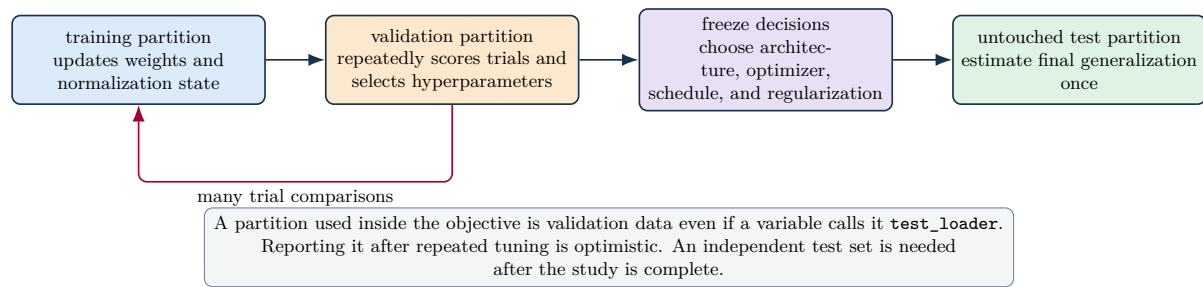


Figure 9: Training updates parameters, validation selects hyperparameters, and an untouched test partition estimates final generalization. [Technical reference](#).

For small data collections, nested cross-validation can estimate the performance of the entire selection procedure. Deep-learning studies often use a fixed train–validation–test split because nested training is expensive, but the conceptual separation remains the same.

Pitfall

Do not tune on a partition and then rename the same score “test accuracy.” Variable names do not change information flow. The deciding question is whether the examples influenced any development decision.

10 Experiment tracking with MLflow-style runs

Hyperparameter tuning generates many related experiments. Experiment tracking records enough context to compare and reproduce them. A useful record contains

- all sampled parameter values;
- validation metric, epoch-level metrics, and final status;
- sampler, pruner, trial number, and random seeds;
- data version and split identity;
- code and dependency versions;
- device, runtime, and relevant resource measurements.

A larger tracked search was described as 50 trials, with one MLflow run intended to correspond to one trial. The dashboard itself displayed “51 matching runs” and “Comparing 51 Runs,” so at least one additional run was present and should not be silently counted as a trial. Its visible accuracy scale reached 0.90642. Parallel-coordinate and scatter views suggested that two or three hidden layers were explored most often, widths near roughly 100–120 were dense, about 40 epochs performed well, useful learning rates appeared around 0.002–0.004, SGD produced strong observed results, and dropout probability 0.1 appeared favorable.

These plots are descriptive, not causal. An adaptive sampler deliberately visits some regions more than others. Dense sampling can reflect both previous outcomes and the sampler’s policy. Hyperparameters also interact: a learning rate that works with SGD may not be appropriate for Adam or RMSprop, and a useful dropout probability can depend on hidden width and epoch count.

Intuition

A tracking dashboard is a laboratory notebook with searchable structure. It helps identify hypotheses for the next controlled study; it does not turn observational trial history into an ablation experiment.

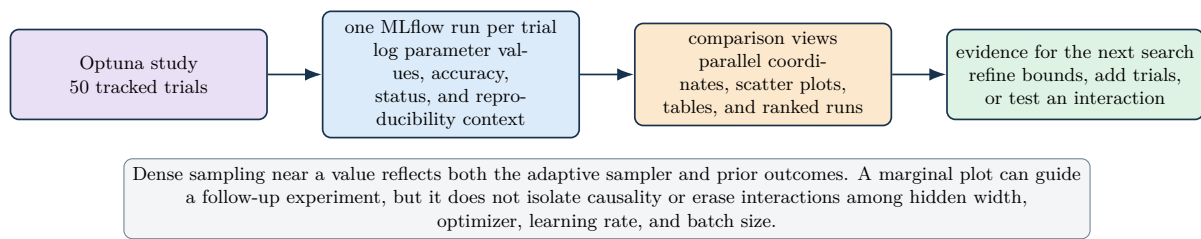


Figure 10: Tracking one run per trial connects parameter values, metrics, comparison views, and the next experimental decision. [Technical reference](#).

11 Executed reconstruction and reproducibility

A deterministic isolated execution validated the complete eight-parameter objective with PyTorch, torchvision, scikit-learn, and Optuna. The run used a stratified 1,500-example Fashion-MNIST subset split into 900 training, 300 validation, and 300 untouched test examples. Four TPE trials executed their sampled schedules of 10–40 epochs on CPU.

The execution verified:

- the complete search ranges, including log-scaled learning rate and weight decay;
- dynamic models with one through three sampled hidden layers;
- batch sizes 32, 64, and 128 across the sampled trials;
- Adam and RMSprop optimizer construction;
- fresh model, loader, optimizer, and state for every trial;
- the best value equaled the maximum of all recorded validation values;
- the manual $L = 2$, $H = 128$ model produced 118,794 parameters and logits shaped [32, 10];
- all 80 round-one architecture choices and 24,000 discrete round-two combinations were counted correctly;
- the expected API, BatchNorm, and matrix-shape failures were raised rather than hidden.

The best local validation value was 0.70, and retraining its selected configuration on the local development subset produced untouched-test accuracy 0.74. These values are smoke-test measurements from a small deterministic subset, not substitutes for the displayed full-data values 0.8933333333333333 and 0.8908333333333334. The host exposed no CUDA accelerator, so GPU timing and a full-data multi-trial reproduction are not claimed.

```

assert len(range(1, 6)) * len(range(8, 129, 8)) == 80
assert 80 * 5 * 5 * 4 * 3 == 24_000
assert study.best_value == max(
    trial.value for trial in study.trials
    if trial.value is not None
)

probe = DynamicANN(784, 10, 2, 128, 0.3)
probe.eval()
with torch.inference_mode():
    logits = probe(torch.zeros(32, 784))
assert logits.shape == (32, 10)
assert sum(p.numel() for p in probe.parameters()) == 118_794
  
```

Listing 15: Make central search and shape invariants executable

Reproducing exact trial metrics additionally requires the same data rows, split, loader order, random states, software stack, hardware, deterministic settings, and trial sequence. A sampler seed alone does

not control every source of randomness inside PyTorch training.

12 Failure modes and debugging checklist

Objective is undefined. Run the function-definition cell before calling `study.optimize`.

Wrong trial-count keyword. Use `n_trials`; the singular `n_trial` raises a type error.

Misspelled categorical API. Use `suggest_categorical`, not the displayed `suggest_categorical`.

Stale dynamic width. Set the next input width to H after appending each hidden layer. Otherwise $[B, H]$ reaches a layer expecting 784 features.

Reused model. Construct a fresh model inside every objective call; never continue training from a previous trial unless warm-starting is explicitly part of the design.

Reused optimizer. Optimizer state belongs to one model and one trial. Recreate it with the sampled family, learning rate, and weight decay.

Loader outside the objective. A sampled batch size has no effect unless the loaders are rebuilt from that value.

Wrong optimization direction. Accuracy requires maximize; loss ordinarily requires minimize.

Validation leakage. A repeatedly consulted partition is validation data, not an untouched final test set.

Wrong target contract. Cross-entropy class indices require shape $[B]$ and integer type `torch.int64`.

Singleton BatchNorm batch. Training-mode `BatchNorm1d` cannot estimate variance from a batch shaped $[1, H]$.

Mode error. Use `model.train()` for fitting and `model.eval()` plus `inference_mode()` for scoring.

Log search with a step. `log=True` and a linear `step` are incompatible in `suggest_float`.

Unfair resource comparison. Epoch count and batch size jointly change the number of optimizer steps; report both compute and score.

Stale configuration variable. Defining `learning_rate=0.01` while hard-coding `lr=0.1` in the optimizer makes the variable inert. Pass the variable explicitly and verify the optimizer parameter group.

Overinterpreted plots. Adaptive sampling frequency and marginal association do not establish causal importance.

Insufficient search budget. Ten trials cannot exhaust even the 80 architecture-only combinations.

13 Review questions

1. Distinguish a learned weight from hidden-layer width, using the inner and outer optimization problems.
2. Define every symbol in $A_{\text{val}}(h)$ and explain what the indicator function computes.
3. Why are there 80 architecture choices when $L \in \{1, \dots, 5\}$ and $H \in \{8, 16, \dots, 128\}$?
4. Reproduce the count of 24,000 discrete round-two combinations before continuous choices.
5. For $L = 2$ and $H = 128$, derive every shape from $[B, 784]$ to $[B, 10]$.
6. Use the parameter-count formula to reproduce 118,794 trainable scalars.
7. Why must `current_width` change after the first hidden layer?

8. Which sampled values require rebuilding the loader, model, optimizer, or schedule?
9. Why is 10^{-3} the geometric midpoint between 10^{-5} and 10^{-1} ?
10. Explain the difference among a study, trial, sampler, objective, and best trial.
11. Why does a partition returned from the objective become validation data even if it is named `test_loader`?
12. What does `direction="maximize"` mean, and when should it be changed to minimize?
13. Why can two adaptive searches with different seeds or budgets return different best configurations?
14. Design an ablation that tests whether dropout probability 0.1 is genuinely better than 0.2 while controlling optimizer and hidden width.
15. What must be logged to reproduce a tracked trial months later?